

Relatório de Auditoria de Segurança

Projeto Quitutes

Data: 01/09/2026

Repositório: Bernawsky/quitutes (branch main)

Total de achados: 8 · Pontos fortes verificados: 8

Escopo auditado

Repositório completo **Bernawsky/quitutes** na branch **main**: todas as rotas de API (`app/api/**/route.ts`), todas as Server Actions (`app/actions/*.ts`), todas as páginas e layouts do App Router, as bibliotecas de acesso a dados (`lib/pedidos-api.ts`, `lib/pousadas-api.ts`), configuração de deploy (`vercel.json`), o serviço auxiliar `whatsapp-service/`, arquivos de ambiente/documentação (`.env.example`, `README.md`), o histórico completo do Git (58 commits) e — via acesso direto ao banco (Supabase MCP) — todas as políticas de Row Level Security e o código-fonte de todas as funções `SECURITY DEFINER` do projeto.

Nota metodológica — mapeamento das categorias para a stack

Stack detectada

Next.js 16 (App Router, TypeScript) + Supabase (Postgres, Auth, Row Level Security) — sem ORM tradicional, o cliente `@supabase/supabase-js` fala direto com o banco. Frontend em React 19 com `@base-ui/react`. Deploy na Vercel, sem Docker, CI, Helm ou Terraform no repositório.

1. Banco sem tranca → RLS do Supabase

O mecanismo de isolamento por inquilino/dono é inteiramente via Row Level Security do Postgres. Cada tabela foi inspecionada com `pg_policies` ao vivo no banco de produção, comparando a política com o que o código da aplicação assume.

2. Permissão no navegador → guards de servidor

Como não há um middleware de autorização central, cada página/Server Action privilegiada precisa chamar explicitamente um guard (`exigirAdminServer()`, `exigirEquipeServer()` ou um `exigirAdmin()` local). Cada gate de UI foi cruzado com o guard correspondente do lado servidor.

3. IDOR → funções de acesso a dados por ID

Sem rotas REST próprias de CRUD — as mutações saem do componente cliente direto pro Supabase. A auditoria percorreu toda função de `lib/pedidos-api.ts` e `lib/pousadas-api.ts` que recebe um ID, além das rotas customizadas em `app/api/` e da única rota dinâmica do projeto.

4. Chaves expostas → grep completo + histórico Git

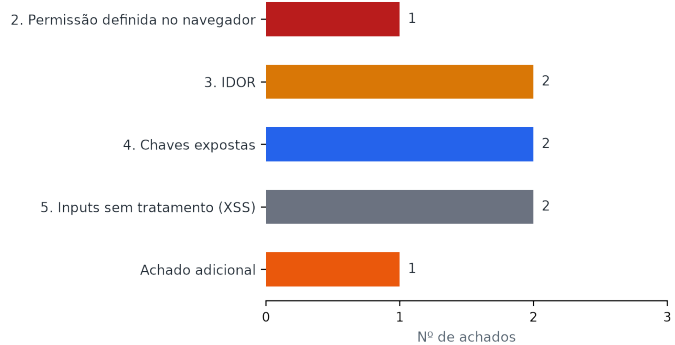
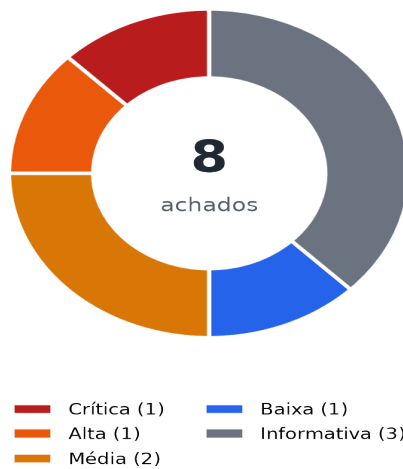
Varredura de padrões de segredo em todo o código-fonte, configs, o serviço `whatsapp-service/`, `.env.example`, `README.md` e nos 58 commits do histórico do Git.

5. Inputs sem tratamento → XSS

Varredura por `dangerouslySetInnerHTML`, `innerHTML`, `eval/new Function`, hrefs controlados por usuário, e checagem de bibliotecas de sanitização instaladas vs. pontos onde HTML de usuário é de fato renderizado.

Resumo executivo

● Crítica: 1 ● Alta: 1 ● Média: 2 ● Baixa: 1 ● Informativa: 3



O achado crítico (A1) e o de severidade alta (A2) concentram o risco real do sistema hoje: uma checagem de papel ausente numa Server Action, e uma checagem de segredo que vira opcional se a variável de ambiente não estiver configurada. Os demais achados são de defesa em profundidade ou informativos — não representam falhas exploráveis nas condições atuais de configuração.

Pontos fortes (evidência de cobertura da auditoria)

- ✓ **RLS granular e correta em todas as 11 tabelas**
Todas as políticas de Row Level Security foram lidas diretamente do banco em produção (`pg_policies`). O isolamento é feito por `pousadas.auth_user_id = auth.uid()` (`pousadas`) e por `user_roles.role` (`admin/operador`) — sem gaps encontrados.
- ✓ **Rotas de notificação nunca confiam no corpo da requisição**
`/api/notificar/cancelamento` e `/api/notificar/evento` sempre releem o pedido do banco através de um client com sessão do próprio usuário (respeitando RLS) antes de montar qualquer mensagem — corrige um padrão que, se copiado do corpo da requisição, seria uma falha clássica de forjar alertas.
- ✓ **Inscrição de push usa o ID de sessão do servidor, nunca do body**
`/api/push/subscribe` grava `user.id` obtido de `supabase.auth.getUser()`;
`/api/push/unsubscribe` usa um client com RLS ativa, então só é possível apagar a própria inscrição.
- ✓ **Formulário público de feedback valida posse antes de gravar**
`/api/feedback` confere o formato do token (regex de UUID) e que a unidade selecionada pertence de fato ao pedido do token ou à pousada escolhida, antes do insert.
- ✓ **Token de feedback não é adivinhável**
`app/feedback/[token]/page.tsx` usa um token com formato UUID (~122 bits de entropia) — sem risco de enumeração para acessar o link de avaliação de outro pedido.
- ✓ **Toda página privilegiada chama seu guard de servidor**
`/administracao`, `/leitor`, `/vouchers` e todo o grupo `/metricas/*` (via layout) chamam `exigirAdminServer()`/`exigirEquipeServer()` — nenhuma rota protegida ficou sem guard.
- ✓ **Nenhum segredo real hardcoded em lugar nenhum**
Varredura completa do código-fonte, configs, `whatsapp-service/`, `.env.example`, `vercel.json` e nos 58 commits do histórico do Git — os únicos valores "reais" encontrados são a URL do projeto Supabase e a chave pública (`sb_publicable_...`), que são públicas por design.
- ✓ **Nenhum vetor de XSS explorável**
Todo texto vindo de usuário passa por escaping automático do JSX, por mensagens de WhatsApp em texto plano (sem interpretação de markup), ou por texto não-interpretado em PDF/CSV.

Pontos fracos — tabela resumo dos achados

Severidade	Arquivo:linha	Descrição
Crítica	app/actions/metricas.ts:15-26	[A1] Server Action de fechamento de período sem checagem de papel (role)
Alta	app/api/cron/fechar-metricas/route.ts:8-14 / 7-11 / 7-11	[A2] Autenticação do cron vira opcional se a variável de ambiente não estiver setada
Média	lib/pedidos-api.ts:144-161	[A3] Editar/cancelar pedido não reforça a posse da pousada na própria query
Média	lib/pedidos-api.ts:salvarPedido	[A4] pousada_id do novo pedido vem do chamador, não é derivado da sessão
Baixa	whatsapp-service/README.md:21	[A5] Número de WhatsApp real do negócio documentado em texto puro
Informativa	Funções SECURITY DEFINER no banco: criar_pousada:–	[A6] Advisor do Supabase sinaliza RPCs SECURITY DEFINER chamáveis por qualquer authenticated
Informativa	package.json / todo o repositório:–	[A7] Nenhuma biblioteca de sanitização instalada
Informativa	lib/export-pedidos.ts:16-19	[A8] Exportação CSV não neutraliza caracteres de início de fórmula

Achados detalhados

Crítica

[A1] Server Action de fechamento de período sem checagem de papel (role)

Categoria: 2. Permissão definida no navegador

Arquivo: app/actions/metricas.ts — linhas 15–26

A Server Action `fecharPeriodo()` é chamada pelo botão "Exportar" dentro da área administrativa (`components/exportar-dialog.tsx`), mas o único controle de acesso dentro da própria função é `if (!user) throw` — confere que existe uma sessão autenticada, mas nunca confere o papel (role) desse usuário.

Isso contrasta diretamente com as funções irmãs do mesmo padrão de projeto — `convidarAdmin()` e `listarAdministradores()` (`app/actions/administracao.ts`) e `definirBuffetAtivo()` (`app/actions/buffet.ts`) — todas chamam um `exigirAdmin()` antes de executar. `fecharPeriodo()` ficou de fora desse padrão.

Uma Server Action do Next.js é um endpoint HTTP próprio, alcançável diretamente por qualquer sessão autenticada independente de qual página React chamou a função — inclusive uma conta de pousada comum, que nunca deveria acessar nada em `/metricas`.

Trecho de código

```
export async function fecharPeriodo(input: {
  periodo: string
  rotuloPeriodo: string
  totalPedidos: number
  ranking: Ranking[]
  pedidos: Pedido[]
}) {
  const supabase = await createServerSupabaseClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error("Não autenticado.")
  // <- nunca confere se user.role === 'admin'
  ...
  const { error } = await supabase.from("metricas_exportadas").insert({ ... })
  const relatorio = await enviarRelatorioPorWhatsapp(input) // dispara WhatsApp pros admins
```

Explorabilidade: Qualquer conta de pousada autenticada pode chamar essa Server Action diretamente (o ID da ação é descoberto no bundle JS do cliente) com valores de pedidos e ranking forjados — a função nunca revalida esses dados contra o banco. O resultado: linhas falsas em `metricas_exportadas` e uma mensagem de WhatsApp forjada disparada para os números reais dos administradores.

Recomendação: Adicionar o mesmo `exigirAdmin()` usado em `app/actions/administracao.ts` e `app/actions/buffet.ts` logo no início de `fecharPeriodo()`, e revalidar pedidos/ranking a partir do banco em vez de confiar no que o cliente envia.

Alta**[A2] Autenticação do cron vira opcional se a variável de ambiente não estiver setada****Categoria:** Achado adicional — controle de acesso quebrado (dependente de ambiente)**Arquivo:** app/api/cron/fechar-metricas/route.ts, lembrete-pendencias/route.ts, relatorio-semanal/route.ts — linhas 8–14 / 7–11 / 7–11

As três rotas de cron job só validam o segredo se a variável de ambiente CRON_SECRET estiver definida — o padrão é `if (secret) { ... checa Authorization ... }`. Se essa variável não estiver configurada no projeto Vercel (esquecimento de configuração, novo ambiente de preview, etc.), o bloco inteiro de checagem é pulado e a rota fica completamente pública.

Trecho de código

```
const secret = process.env.CRON_SECRET
if (secret) {
  const auth = request.headers.get("authorization")
  if (auth !== `Bearer ${secret}`) {
    return NextResponse.json({ error: "unauthorized" }, { status: 401 })
  }
}
// se CRON_SECRET não existir, chega aqui sem nenhuma checagem
```

Explorabilidade: Depende de configuração de ambiente: só é explorável se CRON_SECRET não estiver setado na Vercel. Se estiver configurado (o que parece ser o caso hoje em produção, a julgar pelo `.env.example`), a rota está protegida. O risco é a ausência de uma falha segura (fail-closed): a configuração errada vira silenciosamente uma rota pública que dispara WhatsApp para os administradores e grava no banco via chave de serviço, sem nenhum aviso.

Recomendação: Trocar para fail-closed: se CRON_SECRET não estiver definido, retornar 500/401 imediatamente em vez de pular a checagem — nunca permitir que a ausência da variável vire "sem autenticação".

Média**[A3] Editar/cancelar pedido não reforça a posse da pousada na própria query****Categoria:** 3. IDOR (defesa em profundidade)**Arquivo:** lib/pedidos-api.ts — linhas 144–161 (editarPedidoPousada) e 164–178 (cancelarPedidoPousada)

Essas duas funções filtram a atualização só por `.eq("id", input.id).eq("status", "ativo")` — sem um `.eq("pousada_id", ...)` explícito, diferente de `getMeusPedidos()` (linha 136), que tem esse filtro redundante além da RLS.

A política de UPDATE da tabela pedidos foi inspecionada ao vivo no banco (via `pg_policies`) e confirma corretamente a posse (`EXISTS (... p.auth_user_id = auth.uid())`) — hoje a proteção real existe e está certa. O ponto é que o código da aplicação não tem uma segunda camada: se essa política de RLS um dia regredir (uma migration futura, um erro de revisão), essas duas funções passam a aceitar qualquer id de pedido — e os IDs são inteiros sequenciais, não UUID, portanto adivinháveis por enumeração simples.

Trecho de código

```
export async function cancelarPedidoPousada(input: { id: number; motivo?: string }) {
  const { error } = await supabase
    .from("pedidos")
    .update({ status: "cancelado", motivo_cancelamento: input.motivo ?? null, ... })
    .eq("id", input.id)
    .eq("status", "ativo")
  // sem .eq("pousada_id", ...) — depende 100% da RLS
```

Explorabilidade: Não explorável hoje (a política de RLS cobre corretamente o caso). É um achado de resiliência/defesa em profundidade, não uma vulnerabilidade ativa.

Recomendação: Adicionar `.eq("pousada_id", pousadaIdDaSessao)` nessas duas queries como camada extra, do mesmo jeito que já é feito em `getMeusPedidos()`.

Média

[A4] pousada_id do novo pedido vem do chamador, não é derivado da sessão**Categoria:** 3. IDOR (defesa em profundidade)**Arquivo:** lib/pedidos-api.ts — linhas salvarPedido (17–48) e salvarPedidoBuffet (51–78)

As duas funções de criação de pedido recebem pousadaId como parâmetro comum vindo do componente React, em vez de o servidor derivar esse valor a partir da sessão autenticada. A política de INSERT da tabela (inspecionada ao vivo) já confere corretamente que o pousada_id enviado pertence ao usuário logado — então não é explorável hoje —, mas é o mesmo padrão de risco do achado A3: uma única linha de defesa, inteiramente na RLS.

Trecho de código

```
export async function salvarPedido(input: {  
  ...  
  pousadaId: string // vem do componente, não da sessão  
  pousada: string  
  dataPedido?: string  
}) { ... }
```

Explorabilidade: Não explorável hoje — mitigado pela política de INSERT da RLS, já verificada.**Recomendação:** Considerar mover a resolução de pousadaId para o lado servidor (a partir da sessão), reduzindo a superfície de confiança no cliente.

Baixa

[A5] Número de WhatsApp real do negócio documentado em texto puro**Categoria:** 4. Chaves expostas (dado sensível, não é credencial)**Arquivo:** whatsapp-service/README.md — linhas 21

O README do serviço auxiliar de WhatsApp cita, como instrução de pareamento, o número real usado para rodar a automação ("escaneie com o WhatsApp do número +55 32 98427-5356"). Não é uma chave/segredo de autenticação, mas é uma informação operacional sensível documentada em um arquivo versionado no Git.

Trecho de código

```
"escaneie com o WhatsApp do número +55 32 984275356"
```

Explorabilidade: Não é uma falha de acesso ao sistema — é exposição de PII/dado operacional que pode viabilizar engenharia social ou spam direcionado a esse número.**Recomendação:** Substituir por um placeholder genérico na documentação (ex.: "o número da conta configurada").

Informativa

[A6] Advisor do Supabase sinaliza RPCs SECURITY DEFINER chamáveis por qualquer authenticated

Categoria: 4. Chaves expostas (hardening, não vulnerabilidade)

Arquivo: Funções SECURITY DEFINER no banco: `criar_pousada`, `redefinir_senha_pousada` — linhas —

O linter de segurança nativo do Supabase aponta que essas duas funções podem ser chamadas via `/rest/v1/rpc/...` por qualquer usuário autenticado, já que são SECURITY DEFINER e não têm REVOKE EXECUTE explícito.

Ao ler o código-fonte real das funções direto no banco (`pg_proc.prosrc`), ambas começam com `IF NOT EXISTS (... role = 'admin') THEN RAISE EXCEPTION` — ou seja, chamar como não-admin apenas retorna erro. O alerta do advisor é, na prática, falso positivo.

Trecho de código

```
-- início de ambas as funções, confirmado via pg_proc.prosrc:
IF NOT EXISTS (SELECT 1 FROM public.user_roles
               WHERE user_id = auth.uid() AND role = 'admin'::app_role) THEN
  RAISE EXCEPTION 'Acesso negado: apenas administradores.';
END IF;
```

Explorabilidade: Não explorável — check de admin interno já bloqueia não-admins.

Recomendação: Hardening opcional: `REVOKE EXECUTE ... FROM authenticated` nessas funções, como segunda camada, para que o advisor pare de sinalizar e a superfície fique explicitamente fechada mesmo sem depender do check interno.

Informativa

[A7] Nenhuma biblioteca de sanitização instalada

Categoria: 5. Inputs sem tratamento (risco latente, não vulnerabilidade)

Arquivo: `package.json` / todo o repositório — linhas —

Não há `dompurify`, `sanitize-html` ou equivalente nas dependências. Hoje isso não é um problema — a varredura completa do repositório encontrou só um uso de `dangerouslySetInnerHTML` (`components/ui/chart.tsx:95-109`), e ele injeta apenas variáveis CSS a partir de uma configuração estática de gráfico, nunca de dado de usuário. Todo texto de usuário (nome de pousada, saudação, comentário de feedback, motivo de cancelamento) passa por interpolação JSX comum, que o React escapa automaticamente.

Trecho de código

```
grep -r "dangerouslySetInnerHTML\|innerHTML\|eval(\|new Function(" app/ components/ lib/ hooks/
# → 1 resultado, não relacionado a dado de usuário
```

Explorabilidade: Sem vetor de XSS explorável hoje. Risco só se surgir uma futura renderização de HTML/markdown vindo de usuário sem essa lib.

Recomendação: Nenhuma ação imediata. Se algum dia for necessário renderizar HTML/markdown de usuário, instalar e usar uma lib de sanitização nesse ponto específico.

Informativa

[A8] Exportação CSV não neutraliza caracteres de início de fórmula

Categoria: 5. Inputs sem tratamento (teórico)

Arquivo: lib/export-pedidos.ts — linhas 16–19 (csvCampo), 31–61

A função de escape do CSV (csvCampo) trata corretamente aspas, ponto-e-vírgula e quebras de linha, mas não prefixa campos que começam com =, +, - ou @. Se um nome de pousada ou observação começar com um desses caracteres e o CSV for aberto no Excel/Sheets, o programa pode interpretar o campo como fórmula ("CSV injection" clássico) — mas isso não afeta o sistema Quitutes em si, só quem abrir o arquivo exportado numa planilha.

Trecho de código

```
function csvCampo(v: string) {  
  // escapa aspas;/quebra de linha, mas não =/+/-/@  
  ...  
}
```

Explorabilidade: Teórico, depende de um nome de pousada/comentário malicioso E de abrir o CSV numa planilha vulnerável a essa técnica.

Recomendação: Prefixar com apóstrofo (') campos que comecem com =, +, -, @ antes de escrever no CSV.

Recomendações priorizadas

Prioridade	Recomendação	Onde	Referência
P1	Adicionar checagem de papel (admin) em fecharPeriodo()	app/actions/metricas.ts	Achado A1 — crítico, corrige o gap mais sério encontrado.
P1	Tornar a checagem do CRON_SECRET fail-closed	app/api/cron/*/route.ts	Achado A2 — evita que uma configuração ausente vire rota pública.
P2	Reforçar posse (pousada_id) nas queries de edição/cancelamento	lib/pedidos-api.ts	Achado A3 — defesa em profundidade além da RLS já correta.
P2	Derivar pousada_id da sessão em vez do parâmetro do cliente	lib/pedidos-api.ts	Achado A4 — mesma linha de raciocínio do A3.
P3	Remover o número de WhatsApp real da documentação	whatsapp-service/README.md	Achado A5 — reduz exposição de PII em texto versionado.
P3	Revogar EXECUTE das RPCs para authenticated, além do check interno	Banco (Postgres)	Achado A6 — hardening, já protegido pela checagem interna.
P3	Escapar caracteres de fórmula na exportação CSV	lib/export-pedidos.ts	Achado A8 — proteção teórica contra CSV injection no Excel/Sheets.

Issues para o GitHub

Texto pronto em Markdown para copiar e colar diretamente na criação de uma issue no GitHub. Achados triviais relacionados foram agrupados numa única issue para não gerar spam.

Issue 1 de 4

--- ISSUE 1 ---

[Segurança] Server Action de fechamento de período sem checagem de papel (admin)

Labels sugeridas: security, crítica

Problema

A Server Action `fecharPeriodo()` em `app/actions/metricas.ts` (linhas 15-26) só confere se existe uma sessão autenticada (`if (!user) throw`), mas nunca confere se o usuário tem o papel `admin`. Isso é inconsistente com o resto do projeto: `convidarAdmin()` / `listarAdministradores()` (`app/actions/administracao.ts`) e `definirBuffetAtivo()` (`app/actions/buffet.ts`) chamam um `exigirAdmin()` antes de executar qualquer coisa privilegiada.

Server Actions do Next.js são endpoints HTTP próprios e alcançáveis diretamente por qualquer sessão autenticada, independente de qual página React chamou a função.

Evidência

```
`app/actions/metricas.ts:15-26`
``ts
export async function fecharPeriodo(input: {
  periodo: string
  rotuloPeriodo: string
  totalPedidos: number
  ranking: Ranking[]
  pedidos: Pedido[]
}) {
  const supabase = await createServerSupabaseClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) throw new Error("Não autenticado.")
  // nunca confere user.role === 'admin'
  ...
}
```

Impacto

Qualquer conta de pousada autenticada pode chamar essa Server Action diretamente com ``pedidos`/`ranking`` forjados, gravando linhas falsas em `metricas_exportadas` e disparando uma mensagem de WhatsApp forjada para os números reais dos administradores.

Sugestão de correção

Adicionar a mesma checagem `exigirAdmin()` usada em `app/actions/administracao.ts` e `app/actions/buffet.ts` logo no início de `fecharPeriodo()`, e revalidar `pedidos`/`ranking`` a partir do banco em vez de confiar no que o cliente envia.

Critérios de aceite

- [] `fecharPeriodo()` chama um guard equivalente a `exigirAdmin()` antes de qualquer efeito colateral
- [] Chamar a action autenticado como uma conta de pousada (não-admin) retorna erro e não grava nada

- [] `pedidos`/`ranking` usados no relatório são revalidados/buscados do banco, não confiam cegamente no payload do cliente

--- FIM ISSUE 1 ---

Issue 2 de 4

--- ISSUE 2 ---

[Segurança] Checagem do CRON_SECRET vira opcional se a variável não estiver configurada

****Labels sugeridas:**** security, alta

Problema

As três rotas de cron (`app/api/cron/fechar-metricas/route.ts`, `lembrete-pendencias/route.ts`, `relatorio-semanal/route.ts`) só validam o header `Authorization` se `process.env.CRON\_SECRET` estiver definido: `if (secret) { ... }`. Se a variável não estiver configurada no ambiente (esquecimento, novo ambiente de preview, etc.), a checagem inteira é pulada e a rota fica pública.

Evidência

```
`app/api/cron/fechar-metricas/route.ts:8-14` (mesmo padrão nas outras duas rotas)
`ts
const secret = process.env.CRON_SECRET
if (secret) {
  const auth = request.headers.get("authorization")
  if (auth !== `Bearer ${secret}`) {
    return NextResponse.json({ error: "unauthorized" }, { status: 401 })
  }
}
// se CRON_SECRET não existir, chega aqui sem checagem nenhuma
`
```

Impacto

Numa configuração sem `CRON\_SECRET`, qualquer pessoa na internet pode chamar essas rotas, disparando mensagens de WhatsApp para os números reais dos administradores e escrevendo em `metricas\_exportadas` via client de service-role.

Sugestão de correção

Trocar para fail-closed: se `CRON\_SECRET` não estiver definido, retornar erro (401/500) imediatamente, nunca deixar a ausência da variável significar "sem autenticação".

Critérios de aceite

- [] As 3 rotas de cron retornam erro se `CRON\_SECRET` não estiver definido no ambiente
- [] Uma chamada sem header `Authorization` correto retorna 401 em qualquer ambiente
- [] Documentar em `.env.example`/README que `CRON\_SECRET` é obrigatório em produção

--- FIM ISSUE 2 ---

Issue 3 de 4

--- ISSUE 3 ---

[Segurança] Reforçar defesa em profundidade em edição/criação de pedidos (IDOR)

****Labels sugeridas:**** security, média

Problema

Duas questões relacionadas de defesa em profundidade em `lib/pedidos-api.ts`, ambas mitigadas hoje pela RLS (já verificada e correta), mas sem uma segunda camada no código da aplicação:

1. `editarPedidoPousada` (linhas 144-161) e `cancelarPedidoPousada` (linhas 164-178) filtram só por
`.eq("id", input.id).eq("status", "ativo")``, sem `.eq("pousada_id", ...)`` – diferente de `getMeusPedidos`` (linha 136), que tem esse filtro redundante.
2. `salvarPedido`/`salvarPedidoBuffet` recebem `pousadaId` como parâmetro do chamador em vez de derivá-lo da sessão no servidor.

IDs de pedido são inteiros sequenciais (não UUID), portanto enumeráveis.

Evidência

```
`lib/pedidos-api.ts:144-161`
```ts
export async function editarPedidoPousada(input: { id: number; ... }) {
 const { error } = await supabase
 .from("pedidos")
 .update({ ... })
 .eq("id", input.id)
 .eq("status", "ativo")
 // sem .eq("pousada_id", ...)
}
```
```

Impacto

Nenhum hoje – a política de RLS de UPDATE/INSERT em `pedidos` já confere corretamente a posse via
``pousadas.auth_user_id = auth.uid()``. Isto é puramente uma questão de resiliência: se a RLS um dia
regredir, essas funções passam a aceitar qualquer `id`/`pousadaId`.

Sugestão de correção

Adicionar `.eq("pousada_id", pousadaIdDaSessao)`` em
`editarPedidoPousada`/`cancelarPedidoPousada``, e
considerar derivar `pousadaId` a partir da sessão em vez de aceitá-lo como parâmetro em
`salvarPedido`/`salvarPedidoBuffet``.

Critérios de aceite

- [] `editarPedidoPousada` e `cancelarPedidoPousada` incluem filtro explícito por `pousada\_id`
- [] Testado manualmente: uma pousada não consegue editar/cancelar pedido de outra mesmo manipulando o `id` via console

--- FIM ISSUE 3 ---

Issue 4 de 4

--- ISSUE 4 ---

[Segurança] Hardening: PII em documentação, RPCs sem REVOKE explícito e CSV injection teórico

****Labels sugeridas:**** security, baixa, informativa

Problema

Três itens de hardening de baixa severidade, agrupados por não serem vulnerabilidades ativas:

1. ****PII em documentação**** – `whatsapp-service/README.md:21` expõe o número de WhatsApp real do negócio em texto puro.
2. ****RPCs sem REVOKE explícito**** – `criar\_pousada` e `redefinir\_senha\_pousada` (SECURITY

- DEFINER) são sinalizadas pelo advisor de segurança do Supabase como chamáveis por qualquer `authenticated`. O código-fonte das duas já confirma um `IF NOT EXISTS (...role='admin') THEN RAISE EXCEPTION` no início – não são exploráveis – mas não há `REVOKE EXECUTE` explícito como segunda camada.
3. ****CSV injection teórico**** – `lib/export-pedidos.ts` (`csvCampo`, linhas 16-19) escapa aspas/`;`/quebras de linha, mas não neutraliza campos que começam com `=`, `+`, `-` ou `@`, o que pode virar uma fórmula ao abrir o CSV no Excel/Sheets.

Sugestão de correção

- Substituir o número real no README por um placeholder genérico.
- Rodar `REVOKE EXECUTE ON FUNCTION criar\_pousada, redefinir\_senha\_pousada FROM authenticated;` no banco.
- Prefixar com apóstrofo (`'`) campos do CSV que comecem com `=`, `+`, `-` ou `@`.

Critérios de aceite

- [] README não expõe mais o número de WhatsApp real
- [] `REVOKE EXECUTE` aplicado nas duas RPCs (advisor do Supabase para de sinalizar)
- [] `csvCampo` neutraliza caracteres de início de fórmula

--- FIM ISSUE 4 ---